Chương 2 (Phần 1): Lập trình C an toàn – Quản lý bộ nhớ

2.1 Đánh giá rủi ro – 2.2 Cấu trúc dữ liệu động – 2.3 Lập trình C an toàn

ThS. Vũ Quang Dũng

Khoa Hệ thống thông tin – Trường Đại học Phenikaa

CSE703093 – An toàn phần mềm

Nhắc lại Chương 1

- Chương 1 đã giới thiệu khái niệm bảo mật (CIA, risk), các lớp lỗi phần mềm (buffer overflow, use-after-free...), và nền tảng kiểm thử/kiểm chứng (bao gồm Kripke structure)
- Chương 2 sẽ đi sâu vào **ngôn ngữ C** – nơi phần lớn lỗi bộ nhớ thực sự xảy ra trong thực tế (hệ điều hành, firmware, trình duyệt, hệ thống nhúng)

Nội dung Phần 1

1. 2.1 Đánh giá rủi ro (Risk Assessment)
2. 2.2 Cấu trúc dữ liệu động (Dynamic Data Structures)
3. 2.3 Lập trình C an toàn (Secure C Coding)

Vì sao C vẫn được dùng rộng rãi dù có rủi ro cao?

- Hiệu năng cao, kiểm soát trực tiếp bộ nhớ và phần cứng
- Nền tảng của hệ điều hành (Linux kernel, Windows NT), trình biên dịch, hệ thống nhúng, driver
- Đánh đổi: C **không có** kiểm tra biên mảng, không tự động quản lý bộ nhớ (không có garbage collector) $\Rightarrow$ trách nhiệm an toàn bộ nhớ hoàn toàn thuộc về lập trình viên

Attack Surface của một chương trình C

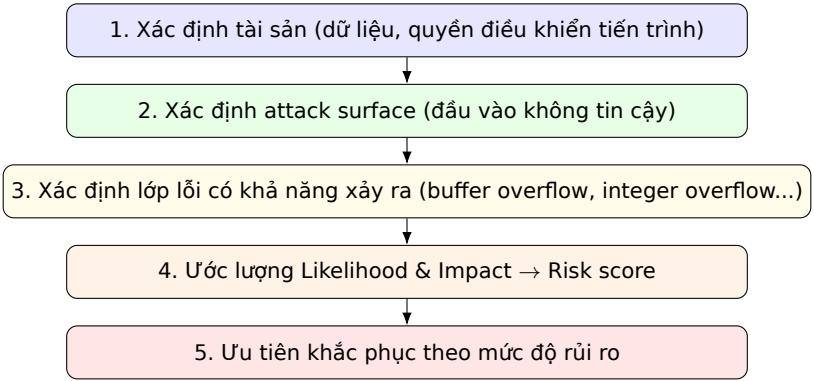
Định nghĩa

Attack Surface: tổng hợp tất cả các điểm mà dữ liệu từ bên ngoài (không tin cậy) có thể đi vào chương trình.

Với chương trình C, các điểm attack surface điển hình:

- Tham số dòng lệnh (`argv`)
- Dữ liệu đọc từ file, mạng (socket), biến môi trường
- Input từ người dùng qua `stdin`, giao diện

Quy trình đánh giá rủi ro cho phần mềm C



Các yếu tố ảnh hưởng đến Likelihood

- **Mức độ tiếp xúc (exposure)**: mã xử lý dữ liệu từ mạng (network-facing) có likelihood cao hơn mã chỉ xử lý file cục bộ
- **Độ phức tạp khai thác**: lỗi dễ kích hoạt (chỉ cần input dài) có likelihood cao hơn lỗi cần điều kiện race hiếm gặp
- **Công cụ khai thác có sẵn**: nếu đã có công cụ/mã khai thác công khai (public exploit), likelihood tăng mạnh

Các yếu tố ảnh hưởng đến Impact

- **Quyền hạn của tiến trình bị ảnh hưởng**: chương trình chạy với quyền `root`/`admin` có impact cao hơn
- **Loại lỗi**: lỗi cho phép *thực thi mã tùy ý* (RCE) có impact cao hơn lỗi chỉ gây crash (DoS)
- **Dữ liệu bị ảnh hưởng**: rò rỉ khóa mật mã/thông tin nhạy cảm có impact cao hơn rò rỉ dữ liệu không quan trọng

Ví dụ đánh giá rủi ro: Hàm phân tích file cấu hình

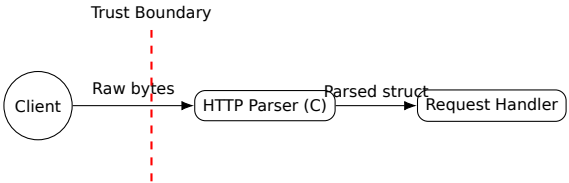
```c
void parse_config(const char *line) {
    char key[32], value[32];
    sscanf(line, "%s = %s", key, value);   // Khong gioi han do dai!
}
```

- Exposure: nếu file cấu hình do người dùng khác (không tin cậy) ghi được → likelihood cao
- Impact: nếu tiến trình chạy quyền root → impact cao (RCE tiềm tàng qua stack overflow)
- ⇒ Risk = Cao, cần ưu tiên sửa ngay bằng `sscanf` có giới hạn độ rộng: `"%31s = %31s"`

Ma trận rủi ro áp dụng cho C: 3 ví dụ

Tình huống	Likelihood	Impact	Risk
Buffer overflow trong parser nhận input từ mạng	Cao	Cao	Cao
Memory leak trong công cụ dòng lệnh nội bộ, chạy 1 lần	Thấp	Thấp	Thấp
Integer overflow hiếm gặp trong thư viện tính toán khoa học nội bộ	Thấp	Trung bình	Trung bình

Threat Modeling cho một ứng dụng C: Ví dụ Web Server đơn giản



HTTP `Parser` là nơi rủi ro cao nhất – xử lý trực tiếp byte thô từ mạng, thường viết bằng C vì lý do hiệu năng.

Case study: Điểm rủi ro cao trong lịch sử – OpenSSH (CVE ví dụ)

- Nhiều lỗ hổng lịch sử trong các daemon mạng viết bằng C (SSH, FTP, DNS server) xuất phát từ lỗi phân tích cú pháp giao thức (protocol parsing)
- Điểm chung: dữ liệu độ dài biến đổi từ mạng được copy vào buffer có kích thước cố định mà không kiểm tra kỹ
- Bài học đánh giá rủi ro: *bất kỳ đoạn mã C nào phân tích dữ liệu từ mạng nên được coi là rủi ro cao mặc định*, trừ khi có bằng chứng ngược lại (kiểm tra kỹ, dùng safe function)

Rủi ro tĩnh vs. Rủi ro động

Rủi ro tĩnh	Có thể xác định chỉ bằng cách đọc mã nguồn (vd: dùng `strcpy` không kiểm tra độ dài)	
Rủi ro động	Chỉ bộc lộ khi chương trình chạy với dữ liệu/điều kiện cụ thể (vd: race condition chỉ xảy ra khi 2 luồng chạy đúng thời điểm)	Đây chính là động lực

để kết hợp **static analysis** (mục 2.5, Chương 3) với **dynamic testing/fuzzing** (Chương 4).

Bài tập: Đánh giá rủi ro

Cho đoạn mã C xử lý input từ một socket TCP mở công khai trên Internet, chạy với quyền root, dùng `gets()` để đọc dữ liệu. Hãy đánh giá Likelihood, Impact, và Risk level.

Đáp án: Đánh giá rủi ro

- Likelihood: **Cao** – `gets()` không giới hạn độ dài, dữ liệu đến từ Internet công khai, rất dễ kích hoạt
- Impact: **Cao** – chạy quyền root $\Rightarrow$ khai thác thành công có thể chiếm toàn quyền hệ thống
- Risk: **Cao (Critical)** – cần vá ngay lập tức; thực tế `gets()` đã bị loại bỏ khỏi chuẩn C11 chính vì nguy hiểm này

Rủi ro theo giai đoạn vòng đời phần mềm

- **Thiết kế**: chọn sai kiến trúc (vd: chạy toàn bộ tiến trình với quyền root) làm tăng impact của mọi lỗi sau này
- **Lập trình**: đây là nơi hầu hết lỗi bộ nhớ C được *tạo ra*
- **Kiểm thử**: nơi phát hiện lỗi qua static/dynamic analysis (Chương 3, 4)
- **Vận hành**: rủi ro còn lại nếu lỗi không được phát hiện trước khi phát hành

Ưu tiên hóa rủi ro trong dự án thực tế

Với nguồn lực hạn chế, không thể sửa mọi lỗi cùng lúc. Cách ưu tiên phổ biến:

1. Lỗi Risk = Cao trên module xử lý dữ liệu từ mạng/người dùng ngoài
2. Lỗi Risk = Cao trên module chạy với quyền cao (root/admin)
3. Lỗi Risk = Trung bình trên module lõi được dùng lại nhiều nơi (thư viện chung)
4. Lỗi Risk = Thấp có thể xếp lịch xử lý sau (technical debt có kiểm soát)

Bài tập: Xác định Attack Surface

Cho một chương trình C xử lý ảnh: đọc file ảnh từ đường dẫn người dùng nhập, phân tích header định dạng ảnh (thư viện tự viết), sau đó ghi kết quả ra file log nội bộ.
Hỏi: Đâu là điểm attack surface rủi ro cao nhất? Vì sao?

Đáp án: Xác định Attack Surface

- Điểm rủi ro cao nhất: **phân tích header định dạng ảnh** – đây là dữ liệu *nội dung file* (không chỉ đường dẫn) có cấu trúc phức tạp, thường do thư viện tự viết (chưa được kiểm chứng kỹ như thư viện chuẩn)
- Đường dẫn file cũng là attack surface (path traversal) nhưng thường dễ kiểm soát hơn bằng validate đường dẫn
- Ghi log nội bộ có rủi ro thấp hơn vì không xử lý dữ liệu ngoài trực tiếp

Liên hệ CVSS chi tiết hơn cho lỗi C

Với một buffer overflow trong hàm phân tích file ảnh (không cần xác thực, khai thác từ xa nếu file được tải lên server):

- Attack Vector: Network hoặc Local (tùy cách file đến được hệ thống)
- Attack Complexity: Thấp (chỉ cần tạo file ảnh có header sai định dạng)
- Privileges Required: None (không cần đăng nhập nếu là chức năng upload công khai)
- $\Rightarrow$ Điểm CVSS thường ở mức **High đến Critical**

Rủi ro trong tái sử dụng thư viện bên thứ ba (C)

- Thư viện C bên thứ ba (nén dữ liệu, phân tích XML/JSON, xử lý ảnh...) có thể chứa lỗ hổng chưa được vá
- Một lỗ hổng trong thư viện dùng chung có thể ảnh hưởng tới *hàng ngàn* sản phẩm phụ thuộc vào nó (tương tự bài học từ Log4Shell ở Chương 1, dù đó là Java)
- Thực hành tốt: theo dõi CVE của các thư viện đang sử dụng, cập nhật phiên bản vá thường xuyên

Tổng kết 2.1

- Đánh giá rủi ro cho C cần xét attack surface cụ thể: input từ mạng, file, dòng lệnh
- Likelihood phụ thuộc exposure và độ dễ khai thác; Impact phụ thuộc quyền hạn và loại lỗi
- Mã phân tích dữ liệu mạng bằng C nên mặc định coi là rủi ro cao

Con trỏ (Pointer) – Nhắc lại nhanh

```c
int x = 10;
int *p = &x;      // p luu dia chi cua x
printf("%d", *p); // *p = 10 (dereference)
*p = 20;          // x bay gio bang 20
```

Con trỏ là nền tảng của mọi cấu trúc dữ liệu động trong C – đồng thời cũng là nguồn gốc chính của các lỗi bộ nhớ đã học ở Chương 1.

Stack vs. Heap

Stack	Cấp phát tự động cho biến cục bộ, kích thước cố định tại compile-time, giải phóng tự động khi hàm kết thúc
Heap	Cấp phát động tại runtime (`malloc`), kích thước linh hoạt, **lập trình viên phải tự giải phóng** bằng `free`

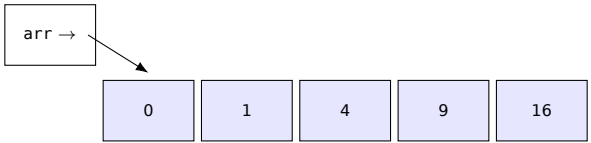
Cấp phát bộ nhớ động: malloc, calloc, realloc, free

`malloc(size)`	Cấp phát `size` byte, nội dung *không xác định*
`calloc(n, size)`	Cấp phát $n \times$ `size` byte, nội dung được **khởi tạo bằng 0**
`realloc(ptr, size)`	Thay đổi kích thước vùng nhớ đã cấp phát, có thể di chuyển vị trí
`free(ptr)`	Giải phóng vùng nhớ, **không** đặt `ptr = NULL` tự động

Ví dụ cấp phát động cơ bản

```c
int *arr = (int *)malloc(10 * sizeof(int));
if (arr == NULL) {
    // Xu ly loi cap phat that bai
    return -1;
}
for (int i = 0; i < 10; i++) arr[i] = i * i;
free(arr);
arr = NULL;   // Thuc hanh tot: tranh dangling pointer
```

Minh họa Heap sau khi cấp phát



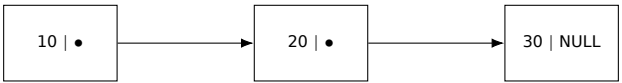
Con trỏ `arr` trỏ tới vùng nhớ heap chứa 10 số nguyên liên tiếp; mỗi phần tử truy cập qua `arr[i]`.

Cấu trúc Linked List – Định nghĩa node

```c
typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node *create_node(int value) {
    Node *n = (Node *)malloc(sizeof(Node));
    if (n == NULL) return NULL;
    n->data = value;
    n->next = NULL;
    return n;
}
```

Minh họa Linked List



Mỗi node cấp phát riêng trên heap, liên kết qua con trỏ `next` – linh hoạt hơn mảng nhưng dễ gây lỗi bộ nhớ nếu quản lý không cẩn thận.

Thêm node vào đầu Linked List

```
Node *push_front(Node *head, int value) {
    Node *n = create_node(value);
    if (n == NULL) return head;  // Cap phat that bai, giu nguyen list
    n->next = head;
    return n;    // Node moi tro thanh head
}
```

Giải phóng toàn bộ Linked List – Đúng cách

```c
void free_list(Node *head) {
    Node *current = head;
    while (current != NULL) {
        Node *next = current->next;  // Luu truoc khi free!
        free(current);
        current = next;
    }
}
```

Lưu ý

Lỗi phổ biến: gọi `free(current)` rồi mới truy cập `current->next` → đây chính là **use-after-free** đã học ở Chương 1.

Lỗi Memory Leak – Ví dụ

```c
void leaky_function() {
    int *data = (int *)malloc(100 * sizeof(int));
    // ... su dung data ...
    return;   // QUEN free(data)! -> memory leak
}
```

Định nghĩa

Memory Leak: bộ nhớ đã cấp phát nhưng không bao giờ được giải phóng, khiến chương trình dần chiếm dụng ngày càng nhiều bộ nhớ hệ thống.

Memory Leak: Không phải lỗi bảo mật trực tiếp nhưng...

- Memory leak thường được coi là lỗi *chất lượng* hơn là lỗ hổng bảo mật trực tiếp
- Tuy nhiên: leak kéo dài có thể dẫn tới cạn kiệt bộ nhớ hệ thống $\Rightarrow$ **Denial of Service** (vi phạm Availability trong CIA)
- Kẻ tấn công có thể chủ động kích hoạt leak lặp đi lặp lại (nếu có thể gọi hàm leaky nhiều lần từ xa) để tấn công DoS có chủ đích

Dangling Pointer

```c
int *get_dangling() {
    int local = 42;
    return &local;   // Tra ve dia chi bien cuc bo!
}   // 'local' bi huy khi ham ket thuc

int *p = get_dangling();
printf("%d", *p);   // Hanh vi KHONG XAC DINH!
```

Lưu ý

Trả về địa chỉ của biến cục bộ (trên stack) sau khi hàm kết thúc tạo ra **dangling pointer** – vùng nhớ đó có thể đã bị ghi đè bởi lời gọi hàm khác.

Cấu trúc (struct) và con trỏ tới struct

```c
typedef struct {
    char name[32];
    int age;
} Person;

Person *p = (Person *)malloc(sizeof(Person));
strncpy(p->name, "An", sizeof(p->name) - 1);
p->age = 20;
free(p);
```

Toán tử `->` truy cập trường của struct thông qua con trỏ – tương đương `(*p).name`.

Mảng động (Dynamic Array)

```c
int *resize_array(int *arr, int old_size, int new_size) {
    int *new_arr = (int *)realloc(arr, new_size * sizeof(int));
    if (new_arr == NULL) {
        // realloc that bai: 'arr' cu VAN CON HOP LE, khong duoc mat no
        return arr;
    }
    return new_arr;
}
```

Lưu ý

Lỗi phổ biến: `arr = realloc(arr, ...)` – nếu realloc thất bại (trả về NULL), con trỏ gốc bị ghi đè và gây **memory leak** vì không còn cách nào giải phóng vùng nhớ cũ.

Giới thiệu ngắn: Cây nhị phân với con trỏ

```c
typedef struct TreeNode {
    int value;
    struct TreeNode *left, *right;
} TreeNode;
```

Cấu trúc cây cũng tiềm ẩn rủi ro bộ nhớ tương tự linked list: giải phóng đệ quy cần cẩn thận tránh use-after-free và đảm bảo giải phóng *cả hai* nhánh con.

Giải phóng cây nhị phân đúng cách

```c
void free_tree(TreeNode *root) {
    if (root == NULL) return;
    free_tree(root->left);
    free_tree(root->right);
    free(root);   // Giai phong node cha SAU CUNG
}
```

Thứ tự *post-order* (giải phóng con trước, cha sau) đảm bảo không truy cập con trỏ đã bị giải phóng.

Công cụ phát hiện lỗi bộ nhớ động: Valgrind

- **Valgrind (memcheck)**: công cụ phân tích động phổ biến, chạy chương trình C và theo dõi mọi thao tác cấp phát/giải phóng bộ nhớ
- Phát hiện: memory leak, use-after-free, đọc/ghi ngoài vùng cấp phát, dùng biến chưa khởi tạo
- Lệnh cơ bản: `valgrind --leak-check=full ./program`
- Đây là công cụ *dynamic analysis* – chỉ phát hiện lỗi trên đường thực thi thực tế đã chạy qua (liên hệ Chương 4)

Bài tập: Tìm lỗi trong đoạn mã Linked List

```c
void remove_first(Node **head) {
    Node *old_head = *head;
    *head = (*head)->next;
    free(old_head);
    printf("Removed: %d\n", old_head->data);  // Loi o day!
}
```

Hãy chỉ ra lỗi và đề xuất cách sửa.

Đáp án: Tìm lỗi trong đoạn mã Linked List

- Lỗi: truy cập `old_head->data` **sau khi** đã `free(old_head)` $\Rightarrow$ **use-after-free** (CWE-416)
- Sửa: lưu giá trị `data` *trước khi* gọi `free`:

```c
int value = old_head->data;
free(old_head);
printf("Removed: %d\n", value);
```

Con trỏ đôi (Pointer to Pointer) và ứng dụng

```
void remove_first(Node **head) {
    // **head cho phep ham thay doi TRUC TIEP con tro head
    // cua ham goi, khong chi ban sao cuc bo
    Node *old_head = *head;
    *head = (*head)->next;
    free(old_head);
}
```

Con trỏ đôi (Node **) cần thiết khi hàm cần *thay đổi chính con trỏ head* ở nơi gọi, không chỉ dữ liệu nó trỏ tới.

Doubly Linked List (Danh sách liên kết đôi)

```c
typedef struct DNode {
    int data;
    struct DNode *prev, *next;
} DNode;
```

- Cho phép duyệt *cả hai chiều*, xóa node ở giữa nhanh hơn (không cần duyệt lại từ đầu)
- Rủi ro bộ nhớ tăng gấp đôi: cần cập nhật *cả* `prev` và `next` đúng cách khi thêm/xóa node, dễ tạo dangling pointer nếu quên một trong hai

Xóa node giữa trong Doubly Linked List

```c
void remove_node(DNode *node) {
    if (node->prev) node->prev->next = node->next;
    if (node->next) node->next->prev = node->prev;
    free(node);
}
```

Phải kiểm tra NULL cho cả `prev` và `next` (trường hợp node là đầu hoặc cuối danh sách) trước khi cập nhật liên kết.

Circular Reference và Memory Leak (giới thiệu)

- Trong C (không có garbage collector), nếu 2 cấu trúc dữ liệu trỏ vòng qua nhau mà không có cơ chế giải phóng rõ ràng, dễ dẫn tới quên giải phóng một trong hai
- Khác với ngôn ngữ có GC (Python, Java) tự phát hiện cycle, C yêu cầu lập trình viên tự thiết kế thứ tự giải phóng chính xác
- Nguyên tắc: xác định rõ "ai sở hữu" (ownership) mỗi vùng nhớ để biết ai chịu trách nhiệm gọi `free`

Ownership Model – Tư duy quan trọng khi quản lý bộ nhớ C

Định nghĩa

Ownership: quy ước rõ ràng về việc *module/hàm nào* chịu trách nhiệm gọi `free` cho một vùng nhớ.

Ví dụ

Quy ước phổ biến: “Hàm nào `malloc` thì hàm đó (hoặc hàm gọi nó theo tài liệu rõ ràng) chịu trách nhiệm `free`”. Vi phạm quy ước này (double free hoặc memory leak) là nguồn lỗi phổ biến nhất trong code base C lớn.

Hash Table đơn giản bằng con trỏ (giới thiệu)

```c
#define TABLE_SIZE 16
typedef struct Entry {
    char *key;
    int value;
    struct Entry *next;  // xu ly va cham (collision) bang chaining
} Entry;

Entry *table[TABLE_SIZE];
```

Mỗi ô trong bảng băm là một linked list (chaining) – kết hợp toàn bộ kiến thức về mảng và con trỏ đã học.

Bài tập: Phát hiện Memory Leak

```c
char *duplicate_string(const char *s) {
    char *copy = malloc(strlen(s) + 1);
    strcpy(copy, s);
    return copy;
}

void process() {
    char *name = duplicate_string("hello");
    printf("%s\n", name);
    // Ham ket thuc o day
}
```

Hãy chỉ ra lỗi memory leak và sửa lại.

Đáp án: Phát hiện Memory Leak

- `duplicate_string` cấp phát bộ nhớ cho copy nhưng `process()` không bao giờ gọi `free(name)`
- Sửa: thêm `free(name);` trước khi hàm `process()` kết thúc
- Bài học: mỗi hàm trả về con trỏ do `malloc` cấp phát cần ghi rõ trong tài liệu rằng *người gọi* chịu trách nhiệm giải phóng

Tổng kết 2.2

- Stack (tự động) vs Heap (thủ công qua malloc/free)
- Linked list, struct, mảng động đều dựa trên con trỏ – nguồn gốc của memory leak, dangling pointer, use-after-free
- Nguyên tắc an toàn: kiểm tra NULL sau malloc, lưu giá trị trước free, đặt NULL sau free, giải phóng theo đúng thứ tự (post-order với cây)
- Valgrind là công cụ thực hành chuẩn để phát hiện lỗi bộ nhớ động

Nguyên tắc lập trình C an toàn – Tổng quan

- Luôn kiểm tra biên (bounds checking) trước khi truy cập mảng/buffer
- Luôn kiểm tra giá trị trả về của `malloc`/`realloc`
- Tránh các hàm chuỗi không an toàn (`gets`, `strcpy`, `sprintf`, `strcat`)
- Khởi tạo mọi biến trước khi dùng
- Kiểm tra tràn số nguyên trước phép toán số học quan trọng

Bảng hàm nguy hiểm và thay thế an toàn

Hàm nguy hiểm	Thay thế an toàn	Lý do
`gets(buf)`	`fgets(buf,n,stdin)`	gets không giới hạn độ dài (đã bị loại khỏi C11)
`strcpy(d,s)`	`strncpy(d,s,n)`	Không kiểm tra kích thước đích
`strcat(d,s)`	`strncat(d,s,n)`	Tương tự strcpy
`sprintf(buf,...)`	`snprintf(buf,n,...)`	Không giới hạn độ dài buffer đích

Ví dụ: sprintf vs snprintf

```c
char msg[32];
// Nguy hiem: neu name dai, tran bo dem
sprintf(msg, "Xin chao, %s!", name);

// An toan hon: gioi han do dai ghi vao msg
snprintf(msg, sizeof(msg), "Xin chao, %s!", name);
```

`snprintf` luôn đảm bảo không ghi vượt quá `sizeof(msg)` byte, kể cả khi `name` rất dài (chuỗi sẽ bị cắt bớt an toàn).

Kiểm tra biên mảng (Bounds Checking)

```c
#define MAX_ITEMS 100
int items[MAX_ITEMS];

int set_item(int index, int value) {
    if (index < 0 || index >= MAX_ITEMS) {
        return -1;  // Tu choi neu ngoai bien
    }
    items[index] = value;
    return 0;
}
```

Luôn kiểm tra *cả hai* chiều: chỉ số âm **và** chỉ số vượt quá kích thước mảng.

Kiểm tra tràn số nguyên (Integer Overflow Check)

```c
#include <limits.h>

int safe_add(int a, int b, int *result) {
    if ((b > 0 && a > INT_MAX - b) ||
        (b < 0 && a < INT_MIN - b)) {
        return -1;  // Se tran so, tu choi phep tinh
    }
    *result = a + b;
    return 0;
}
```

Kiểm tra *trước* khi thực hiện phép cộng, không kiểm tra *sau* (vì lúc đó tràn số đã xảy ra).

Kiểm tra kích thước cấp phát (tránh Integer Overflow trong malloc)

```c
// Nguy hiem: n * sizeof(int) co the tran so neu n rat lon
int *arr = malloc(n * sizeof(int));

// An toan hon: dung ham kiem tra tran, vi du calloc
// (calloc kiem tra tran so nhan noi bo theo chuan C)
int *arr_safe = calloc(n, sizeof(int));
```

`calloc(n, size)` được chuẩn C yêu cầu tự kiểm tra tràn số khi tính $n \times$ size, an toàn hơn tự nhân thủ công.

Validate mọi input trước khi xử lý

```c
int process_age(const char *input) {
    char *endptr;
    long age = strtol(input, &endptr, 10);
    if (endptr == input || *endptr != '\0') {
        return -1;  // Khong phai so hop le
    }
    if (age < 0 || age > 150) {
        return -1;  // Ngoai pham vi hop ly
    }
    return (int)age;
}
```

`strtol` an toàn hơn `atoi` vì cho phép kiểm tra lỗi phân tích cú pháp.

Assertions và Defensive Programming

```c
void process_buffer(char *buf, size_t len) {
    assert(buf != NULL);
    assert(len > 0 && len <= MAX_BUFFER_SIZE);
    // ... xu ly buf ...
}
```

- `assert` giúp phát hiện lỗi lập trình sớm trong môi trường phát triển/test
- Lưu ý: `assert` thường bị **tắt** trong bản release (biên dịch với `NDEBUG`) $\Rightarrow$ không được dùng thay cho kiểm tra input thực sự trong mã production

Bảo vệ ở tầng biên dịch/hệ điều hành (giới thiệu)

Stack Canary	Giá trị kiểm tra được đặt trước địa chỉ trả về; nếu bị ghi đè do buffer overflow, chương trình phát hiện và dừng ngay
ASLR	*Address Space Layout Randomization*: ngẫu nhiên hóa địa chỉ bộ nhớ, gây khó khăn cho kẻ tấn công khi dự đoán địa chỉ
NX bit / DEP	Đánh dấu vùng nhớ dữ liệu là *không thể thực thi*, ngăn chặn thực thi mã được tiêm vào qua lỗi tràn bộ đệm
FORTIFY_SOURCE	Cờ biên dịch GCC/Clang thêm kiểm tra biên tự động cho một số hàm chuỗi

Lưu ý: Các cơ chế bảo vệ không thay thế mã nguồn an toàn

Lưu ý

Stack canary, ASLR, NX bit là các lớp phòng thủ *bổ sung* (defense in depth, đã học ở Chương 1) – chúng làm giảm khả năng khai thác thành công, nhưng **không sửa được lỗi gốc**. Lập trình an toàn từ đầu vẫn là biện pháp quan trọng nhất.

CERT C Coding Standard – Một số quy tắc tiêu biểu

STR31-C	Đảm bảo buffer đủ lớn để lưu chuỗi và ký tự null-terminator
MEM31-C	Giải phóng bộ nhớ đúng một lần (tránh double free)
MEM30-C	Không truy cập bộ nhớ đã giải phóng (tránh use-after-free)
INT32-C	Đảm bảo phép toán số nguyên không bị tràn
ARR30-C	Không truy cập mảng ngoài phạm vi đã cấp phát

MISRA C – Ứng dụng trong hệ thống nhúng/ô tô

- Bộ quy tắc nghiêm ngặt cho C, ban đầu phát triển cho ngành ô tô, nay dùng rộng rãi trong hệ thống nhúng an toàn cao
- Một số quy tắc tiêu biểu: cấm dùng con trỏ hàm phức tạp, cấm chuyển đổi kiểu ngầm định nguy hiểm, bắt buộc kiểm tra mọi giá trị trả về của hàm quan trọng
- Triết lý chung: giảm thiểu *hành vi không xác định* (undefined behavior) – nguồn gốc của rất nhiều lỗ hổng bảo mật trong C

Undefined Behavior (UB) trong C – Vì sao nguy hiểm?

Định nghĩa

Undefined Behavior: hành vi mà chuẩn C *không quy định* kết quả – trình biên dịch được phép làm *bất cứ điều gì*, kể cả tối ưu hóa theo cách gây lỗi bảo mật.

Ví dụ

Truy cập mảng ngoài phạm vi, tràn số nguyên có dấu (signed overflow), dùng con trỏ sau khi free – tất cả đều là UB. Trình biên dịch có thể loại bỏ hẳn đoạn code kiểm tra lỗi nếu nó ”chứng minh” được rằng UB không bao giờ xảy ra (dù thực tế có thể xảy ra)!

Ví dụ UB gây mất tác dụng của kiểm tra bảo mật

```
char buf[16];
int offset = get_offset();  // Co the la so am rat lon
if (buf + offset < buf) {   // Kiem tra tran con tro
    return -1;  // ERROR: hanh vi nay la UB, compiler co the LOAI BO!
}
process(buf + offset);
```

So sánh con trỏ vượt phạm vi mảng là UB – trình biên dịch tối ưu hóa (GCC/Clang -O2) có thể xóa bỏ hoàn toàn câu lệnh `if` này vì ”giả định” nó luôn đúng.

Compiler Warnings – Tuyến phòng thủ đầu tiên

```
gcc -Wall -Wextra -Werror -fsanitize=address,undefined \
    -o program program.c
```

- `-Wall -Wextra`: bật cảnh báo biên dịch mở rộng (biến chưa dùng, so sánh kiểu nguy hiểm...)
- `-Werror`: biến mọi warning thành error, buộc sửa trước khi build thành công
- `-fsanitize=address`: (AddressSanitizer) phát hiện lỗi bộ nhớ ngay khi chạy, nhanh và chi tiết hơn Valgrind trong nhiều trường hợp
- `-fsanitize=undefined`: (UBSan) phát hiện undefined behavior khi chạy

AddressSanitizer (ASan) – Ví dụ output

```
==12345==ERROR: AddressSanitizer: heap-buffer-overflow
READ of size 4 at 0x602000000014 thread T0
    #0 in main program.c:8
0x602000000014 is located 4 bytes after
    a 10-byte region [0x602000000000,0x60200000000a)
```

ASan chỉ ra *chính xác dòng mã* và *loại lỗi* (heap-buffer-overflow), giúp debug nhanh hơn rất nhiều so với crash không rõ nguyên nhân.

Immutable data và const-correctness

```c
void print_name(const char *name) {  // 'const' bao dam khong sua doi
    printf("%s\n", name);
    // name[0] = 'X';  // Loi bien dich neu bo comment - tot!
}
```

Dùng `const` bất cứ khi nào có thể giúp trình biên dịch *tự động* phát hiện các thay đổi ngoài ý muốn – một hình thức ”kiểm chứng” nhẹ ngay tại compile-time.

Bài tập: Xác định UB trong đoạn mã

```
int compute(int a, int b) {
    int result = a + b;  // Neu a, b lon, co the tran so co dau
    return result;
}
```

Nếu a = INT_MAX và b = 1, điều gì xảy ra? Đây có phải Undefined Behavior không?

Đáp án: Xác định UB

- Đúng vậy – tràn số nguyên **có dấu** (signed integer overflow) là **Undefined Behavior** theo chuẩn C (khác với số nguyên *không dấu*, vốn được định nghĩa rõ là ”wrap around”)
- Trình biên dịch có thể tối ưu hóa theo cách giả định điều này không bao giờ xảy ra, dẫn tới kết quả khó lường
- Sửa: dùng kỹ thuật kiểm tra trước khi cộng như slide `safe_add` đã học

Checklist Code Review cho mã C

- ☐ Mọi lời gọi `malloc`/`calloc`/`realloc` đều kiểm tra giá trị trả về
- ☐ Không sử dụng `gets`, `strcpy`, `strcat`, `sprintf` không giới hạn
- ☐ Mọi truy cập mảng đều được kiểm tra biên
- ☐ Mọi `free` đi kèm với việc không truy cập con trỏ đó nữa (và nên đặt về NULL)
- ☐ Phép toán số học có khả năng tràn được kiểm tra trước khi thực hiện

Bài tập: Sửa đoạn mã không an toàn (1)

```
void greet(char *name) {
    char msg[50];
    strcpy(msg, "Hello, ");
    strcat(msg, name);
    printf("%s\n", msg);
}
```

Hãy chỉ ra rủi ro và viết lại phiên bản an toàn.

Đáp án: Sửa đoạn mã (1)

```c
void greet_safe(const char *name) {
    char msg[50];
    snprintf(msg, sizeof(msg), "Hello, %s", name);
    printf("%s\n", msg);
}
```

`snprintf` kết hợp cả nối chuỗi và giới hạn độ dài trong một lời gọi duy nhất, tránh cả hai lỗi `strcpy` và `strcat` cùng lúc.

Bài tập: Sửa đoạn mã không an toàn (2)

```
int *make_array(int n) {
    int *arr = malloc(n * sizeof(int));
    for (int i = 0; i <= n; i++) {   // Loi: <= thay vi <
        arr[i] = 0;
    }
    return arr;
}
```

Hãy chỉ ra *2 lỗi* trong đoạn mã trên.

Đáp án: Sửa đoạn mã (2)

- Lỗi 1: không kiểm tra `arr == NULL` sau `malloc`
- Lỗi 2: điều kiện vòng lặp `i <= n` truy cập `arr[n]` – **vượt quá** vùng nhớ cấp phát (off-by-one, CWE-787 out-of-bounds write)
- Sửa: kiểm tra NULL, và đổi điều kiện thành `i < n`

So sánh: Static Analysis vs. Sanitizers vs. Valgrind

	Khi nào phát hiện	**Chi phí**
Static Analysis	Compile-time, không cần chạy	Nhanh, có false positive
ASan/UBSan	Runtime, cần chạy qua đúng đường lỗi	Chậm hơn 2x, chính xác cao
Valgrind	Runtime, tương tự ASan	Chậm hơn nhiều (10-30x), rất chi tiết

Thực hành tốt: dùng *cả ba* ở các giai đoạn khác nhau – static analysis khi code, sanitizer khi test hàng ngày (CI), Valgrind khi điều tra sâu một lỗi cụ thể.

Fuzzing kết hợp Sanitizer (xem trước Chương 4)

- Biên dịch chương trình với `-fsanitize=address` rồi chạy fuzzer (vd: AFL, libFuzzer) để tự động sinh input
- Khi fuzzer tìm ra input gây lỗi bộ nhớ, ASan sẽ báo lỗi *ngay lập tức và chính xác*, thay vì chương trình chỉ crash mơ hồ
- Đây là kỹ thuật chuẩn công nghiệp để tìm lỗ hổng bộ nhớ trong các dự án C lớn (nhân Linux, trình duyệt, OpenSSL...)

Bài tập tổng hợp: Rà soát đoạn mã C hoàn chỉnh

```c
char *read_line(FILE *f) {
    char buf[256];
    fgets(buf, sizeof(buf), f);
    char *result = malloc(strlen(buf));  // Loi 1?
    strcpy(result, buf);                 // Loi 2?
    return result;
}
```

Hãy tìm *2 lỗi* trong đoạn mã trên (gợi ý: liên quan tới kích thước cấp phát và kiểm tra trả về).

Đáp án: Rà soát đoạn mã C hoàn chỉnh

- Lỗi 1: `malloc(strlen(buf))` thiếu `+ 1` cho ký tự null-terminator ⇒ `strcpy` sau đó sẽ ghi **tràn 1 byte** ra ngoài vùng cấp phát (off-by-one buffer overflow)
- Lỗi 2: không kiểm tra `result == NULL` trước khi `strcpy` ⇒ nếu `malloc` thất bại, chương trình sẽ ghi vào địa chỉ NULL (crash hoặc hành vi không xác định)
- Sửa: `malloc(strlen(buf) + 1)` và kiểm tra NULL trước khi dùng

Vai trò của Chương 2 trong dự án Python

Module `ch2_memsafe` (xây dựng ở phần thực hành) sẽ:

- Quét mã nguồn C tìm các lời gọi hàm nguy hiểm (`strcpy`, `gets`, `sprintf`...) bằng kỹ thuật pattern matching đơn giản
- Mô phỏng một *memory model* tối giản: theo dõi các lệnh `malloc`/`free` trong một "vết thực thi" (execution trace) để phát hiện double-free và use-after-free
- Đây là bước đệm quan trọng trước khi học công cụ phân tích tĩnh chuyên nghiệp hơn ở mục 2.5 và Chương 3

Ví dụ nâng cao: Stack overflow do đệ quy sâu

```
void recurse(int n) {
    char buf[1024];  // Moi lan goi de quy chiem them 1KB stack
    if (n <= 0) return;
    recurse(n - 1);
}
```

Lưu ý

Đệ quy quá sâu (vd: n rất lớn hoặc dữ liệu đầu vào không kiểm soát điều khiển độ sâu đệ quy) có thể làm **tràn stack** (stack overflow) – khác với heap buffer overflow đã học, đây là loại lỗi liên quan tới *kích thước tổng của stack*, không phải một buffer đơn lẻ.

Giải pháp: Chuyển đệ quy thành vòng lặp (Tail Recursion Elimination thủ công)

```
void iterate(int n) {
    char buf[1024];  // Chi cap phat MOT LAN duy nhat
    while (n > 0) {
        n--;
    }
}
```

Với dữ liệu đầu vào không tin cậy điều khiển độ sâu đệ quy (vd: cấu trúc JSON lồng nhau từ người dùng), nên giới hạn độ sâu tối đa tường minh hoặc chuyển sang vòng lặp.

Bài tập: Giới hạn độ sâu đệ quy an toàn

Một hàm phân tích cú pháp biểu thức toán học lồng nhau (dấu ngoặc) bằng đệ quy. Dữ liệu đầu vào đến từ người dùng. Hãy đề xuất một biện pháp phòng ngừa stack overflow.

Đáp án: Giới hạn độ sâu đệ quy an toàn

```c
#define MAX_DEPTH 100
int parse_expr(const char *s, int depth) {
    if (depth > MAX_DEPTH) return -1;  // Tu choi neu qua sau
    // ... de quy voi depth + 1 ...
}
```

Thêm tham số đếm độ sâu và kiểm tra giới hạn tường minh – đây là một dạng *input validation* áp dụng cho cấu trúc đệ quy, không chỉ cho giá trị dữ liệu đơn thuần.

Tổng kết mở rộng: Nguyên tắc chung quản lý bộ nhớ động

1. Mỗi `malloc` nên có đúng một `free` tương ứng, rõ ràng về ownership
2. Không truy cập con trỏ sau khi đã `free` (đặt về NULL để dễ phát hiện lỗi này qua crash sớm thay vì hành vi ngầm)
3. Luôn kiểm tra kết quả của `malloc`/`realloc` trước khi dùng
4. Cẩn trọng với cấu trúc đệ quy/lồng nhau có độ sâu do dữ liệu ngoài kiểm soát

Tổng kết 2.3

- Ưu tiên hàm chuỗi an toàn: `snprintf`, `strncpy`, `fgets` thay cho các hàm không giới hạn độ dài
- Luôn kiểm tra: giá trị trả về malloc, biên mảng, tràn số nguyên trước khi tính toán
- Cơ chế bảo vệ compiler/OS (stack canary, ASLR, NX, FORTIFY_SOURCE) là lớp phòng thủ bổ sung, không thay thế mã nguồn an toàn
- Checklist code review giúp hệ thống hóa việc rà soát các lỗi phổ biến

Case study mở rộng: Lỗi trong một hàm phân tích giao thức mạng

```
void parse_packet(unsigned char *data, int len) {
    unsigned char header_len = data[0];
    unsigned char payload[256];
    memcpy(payload, data + 1, header_len);  // header_len toi da 255, OK?
}
```

- Nhìn thoáng qua có vẻ an toàn (`header_len` tối đa $255 < 256$)
- Nhưng nếu `len` (độ dài thực tế của `data`) nhỏ hơn `header_len + 1`, `memcpy` sẽ **đọc ngoài vùng nhớ nguồn** (out-of-bounds read, CWE-125) – giống cơ chế của Heartbleed đã học ở Chương 1!

Sửa lỗi case study

```c
void parse_packet_safe(unsigned char *data, int len) {
    if (len < 1) return;
    unsigned char header_len = data[0];
    if (len < 1 + header_len) return;  // Kiem tra du du lieu nguon
    unsigned char payload[256];
    memcpy(payload, data + 1, header_len);
}
```

Bài học: phải kiểm tra *cả hai phía* – kích thước đích (destination) **và** kích thước nguồn (source) đều đủ để thao tác an toàn.

Bài tập nhóm: Thiết kế API an toàn

Thiết kế lại API sau để an toàn hơn:

```
// API hien tai (khong an toan)
void copy_data(char *dest, char *src);
```

Gợi ý: API này thiếu thông tin gì để có thể an toàn?

Đáp án: Thiết kế API an toàn

```
// API an toan hon: yeu cau ro rang kich thuoc dich
int copy_data_safe(char *dest, size_t dest_size,
                   const char *src, size_t src_len);
```

- API gốc thiếu thông tin về *kích thước bộ đệm đích*, khiến hàm không thể tự kiểm tra an toàn
- Nguyên tắc thiết kế API an toàn: luôn truyền kèm kích thước buffer, không dựa vào quy ước ngầm (như chuỗi kết thúc bằng `'\0'`)

Thảo luận: Vì sao Rust ngày càng phổ biến thay thế C?

- Ngôn ngữ Rust có *borrow checker* kiểm tra tại compile-time: đảm bảo không có dangling pointer, use-after-free, data race
- Vẫn giữ hiệu năng tương đương C (không có garbage collector runtime)
- Nhiều dự án lớn (Linux kernel module, trình duyệt Firefox) đang dần viết lại một số thành phần bằng Rust để giảm lỗ hổng bộ nhớ
- Tuy nhiên C vẫn thống trị hệ thống nhúng, driver cũ, và mã kế thừa (legacy) – kỹ năng lập trình C an toàn vẫn rất cần thiết

Tự đánh giá nhanh (Self-check) Phần 1

1. Liệt kê 3 hàm chuỗi C nguy hiểm và hàm thay thế an toàn tương ứng
2. Vì sao `realloc` cần được gán vào biến tạm trước khi cập nhật con trỏ gốc?
3. Undefined Behavior khác gì với lỗi thông thường (runtime error có thể dự đoán được)?

Bài tập bổ sung: So sánh rủi ro hai module

Module A: thư viện xử lý JSON dùng nội bộ, chỉ nhận dữ liệu từ tiến trình khác trên cùng máy chủ tin cậy. Module B: thư viện xử lý JSON tương tự, nhưng nhận dữ liệu trực tiếp từ API công khai trên Internet.
Hỏi: Dù có *cùng một lỗi* buffer overflow, rủi ro của 2 module có giống nhau không? Giải thích.

Đáp án: So sánh rủi ro hai module

- **Không giống nhau**, dù lỗi kỹ thuật (technical vulnerability) là như nhau
- Module B có **Likelihood cao hơn nhiều** vì attack surface tiếp xúc trực tiếp với Internet, bất kỳ ai cũng có thể gửi dữ liệu độc hại
- Module A có Likelihood thấp hơn vì chỉ tiếp xúc với môi trường nội bộ đã được tin cậy một phần (dù vẫn cần vá theo nguyên tắc defense in depth)
- Bài học: **rủi ro phụ thuộc ngữ cảnh triển khai**, không chỉ phụ thuộc bản chất kỹ thuật của lỗi

Tài liệu tham khảo thêm cho Phần 1

- CERT C Coding Standard (SEI CMU) – bộ quy tắc lập trình C an toàn đầy đủ
- MISRA C:2012 Guidelines
- OWASP: Buffer Overflow Cheat Sheet
- Google Sanitizers documentation (AddressSanitizer, UndefinedBehaviorSanitizer)

Bài tập lớn nộp cuối buổi (chuẩn bị)

Chọn một trong các cấu trúc dữ liệu đã học (linked list, doubly linked list, hoặc cây nhị phân), viết mã C hoàn chỉnh gồm: tạo, thêm phần tử, xóa phần tử, giải phóng toàn bộ. Sau đó biên dịch với `-fsanitize=address` và chạy thử để xác nhận không có lỗi bộ nhớ.

Chuẩn bị dự án Python (module ch2_memsafe)

Buổi thực hành tiếp theo sẽ bắt đầu xây dựng module `ch2_memsafe` trong `softsec-toolkit`:

- `pattern_scanner.py`: quét mã nguồn C tìm các hàm nguy hiểm đã học (`strcpy`, `gets`, `sprintf`...)
- `memory_trace.py`: mô phỏng theo dõi `malloc`/`free` trong một vết thực thi để phát hiện double-free/use-after-free

Bảng tổng hợp lỗi bộ nhớ đã học trong Chương 2

Lỗi	Nguyên nhân	Cách phòng ngừa
Buffer Overflow	Ghi vượt kích thước buffer	Dùng hàm an toàn, kiểm tra biên
Use-After-Free	Truy cập sau `free`	Đặt NULL sau free, lưu giá trị trước free
Double Free	Gọi `free` 2 lần	Rõ ràng ownership, đặt NULL sau free
Memory Leak	Quên gọi `free`	Ghi rõ ownership trong tài liệu hàm
Integer Overflow	Phép toán vượt giới hạn kiểu	Kiểm tra trước khi tính, dùng `calloc`

Câu hỏi ôn tập nhanh Phần 1

1. Tại sao `calloc(n, size)` an toàn hơn `malloc(n * size)` khi n do người dùng cung cấp?
2. Đâu là khác biệt giữa lỗi bộ nhớ *tĩnh* và *động* trong ngữ cảnh đánh giá rủi ro?
3. Vì sao trả về con trỏ tới biến cục bộ (stack) luôn là lỗi nghiêm trọng?

Kết nối tới Chương 3

Các khái niệm memory model, undefined behavior, và lỗi bộ nhớ đã học ở Chương 2 chính là *đối tượng* mà các công cụ phân tích tĩnh và bounded model checking ở Chương 3 sẽ tự động phát hiện, thay vì phải rà soát thủ công như trong chương này.

Chuyển tiếp sang Phần 2

Phần 2 của Chương 2 sẽ trình bày:

- 2.4 Sắp xếp cấu trúc dữ liệu (Data Structure Alignment)
- 2.5 Một số công cụ phân tích tĩnh để tìm lỗi bảo mật
- 2.6 Tổng kết Chương 2 + hoàn thiện module Python `ch2_memsafe`